HCIA-AI V4.0 Code-Writing & Multi-Blank Workbook

50 High-Yield Code Completion & Matrix Math Questions • Covering Single-Blank, 2–3 Multi-Blank Snippets, PyTorch LeNet & Model Persistence

Curated by: **Umed Jamal Nouri** • **v1.1 (Nov 2026)** • **github.com/UmedJaMala**

EXAM CODE COMPLETION BLUEPRINT & MULTI-BLANK CONVENTIONS

1. Case-Sensitivity & Capitalization
- Method & function names are strictly lowercase: zero_grad, backward, step, fit, predict
- Base classes & loss functions use PascalCase: nn.Module, nn.Cell, nn.CrossEntropyLoss

2. Single & Multi-Blank Rules ([1], [2], [3])
- Real exams include 2–3 blanks inside one code block: Prompt: optimizer.___[1]___() → **Enter only:** zero_grad
- Never duplicate outer parentheses like step()().
- Numeric matrix questions expect bare numbers (e.g., 28, 64, 1).

3. V4.0 Framework Core Focus
- **PyTorch (Primary V4.0):** Uses nn.Module & def forward(self, x):
- **MindSpore (Reference):** Uses nn.Cell & def construct(self, x):
- **Checkpoints:** .ckpt (Protocol Buffers), .pt/.pth, .onnx, .bin

4. Scikit-Learn Trailing Underscores
- Attributes learned during fit() end with an underscore: coef_ (weights), intercept_ (bias), cluster_centers_
- **Data Leakage Rule:** fit_transform() on train data, strictly transform() on test data.

Category 1: PyTorch Deep Learning Training & Workflow (Q1 – Q15)

QUESTION 1 • PyTorch Autograd — 16 Points

In the PyTorch training loop, which method must be called on the optimizer before calculating gradients to clear previously accumulated gradient values?

```
optimizer.____()
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

CORRECT TOKEN(S): `zero_grad (or zero_grad())`

Technical Breakdown & Key Insight: PyTorch accumulates gradients by default across iterations. Calling optimizer.zero_grad() clears existing gradients before computing the new batch gradients.

QUESTION 2 • PyTorch Backpropagation — 16 Points

In PyTorch, which method is invoked on the computed scalar loss tensor to execute backpropagation and calculate the gradients of the loss with respect to all model parameters?

```
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.____()
optimizer.step()
```

CORRECT TOKEN(S): `backward (or backward())`

Technical Breakdown & Key Insight: Calling loss.backward() computes the derivative of the loss with respect to all graph leaves (trainable parameters) using the chain rule.

QUESTION 3 • PyTorch Optimizer Update — 16 Points

In PyTorch, which method of the optimizer is called to update the model parameters (weights and biases) according to the computed gradients?

```
optimizer.zero_grad()
loss.backward()
optimizer.____()
```

CORRECT TOKEN(S): `step (or step())`

Technical Breakdown & Key Insight: optimizer.step() applies the optimization algorithm formula (such as SGD or Adam) to update all weight and bias tensors in-place.

QUESTION 4 • PyTorch Model Inheritance — 16 Points

In PyTorch, all user-defined neural network architectures must inherit from which base class in the torch.nn package?

```
import torch.nn as nn

class LeNet(nn.____):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5)
```

CORRECT TOKEN(S): `Module`

Technical Breakdown & Key Insight: Must be capitalized: Module. Inheriting from nn.Module registers sub-modules and parameters, enabling automatic device movement.

QUESTION 5 • PyTorch Forward Method — 16 Points

In PyTorch, which core method must be implemented inside a custom nn.Module subclass to define the forward computation logic of the network?

```
class CustomClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(128, 10)

    def ____(self, x):
        return self.fc(x)
```

CORRECT TOKEN(S): `forward`

Technical Breakdown & Key Insight: The method is named forward (lowercase). In MindSpore, the equivalent method is named construct — a frequent exam distinction!

QUESTION 6 • PyTorch Inference Mode — 16 Points

During model evaluation and testing in PyTorch, which context manager is used to temporarily deactivate the autograd engine and save memory overhead?

```
model.eval()
with torch.____():
    for images, labels in test_loader:
        outputs = model(images)
```

CORRECT TOKEN(S): `no_grad (or no_grad())`

Technical Breakdown & Key Insight: with torch.no_grad(): disables gradient tracking entirely, speeding up inference and preventing out-of-memory GPU crashes.

QUESTION 7 • PyTorch Evaluation Mode — 16 Points

In PyTorch, which method sets the neural network into evaluation mode, disabling layers such as Dropout and freezing Batch Normalization running statistics?

```
model.____()
with torch.no_grad():
    predictions = model(test_data)
```

CORRECT TOKEN(S): `eval (or eval())`

Technical Breakdown & Key Insight: model.eval() does NOT disable gradients; it modifies Dropout (disabled) and BatchNorm (uses population running stats).

QUESTION 8 • PyTorch Training Mode — 16 Points

In PyTorch, which method sets the neural network back into training mode before starting an epoch of model optimization?

```
model.____()
for inputs, targets in train_loader:
    optimizer.zero_grad()
```

CORRECT TOKEN(S): `train (or train())`

Technical Breakdown & Key Insight: model.train() re-enables stochastic dropout layers and batch normalization updates for the next training pass.

QUESTION 9 • PyTorch Dataset Indexing — 16 Points

In PyTorch, when building a custom Dataset by subclassing torch.utils.data.Dataset, which magic method must be overridden to support indexing and fetch a sample by index?

```
from torch.utils.data import Dataset

class ImageDataset(Dataset):
    def __init__(self, data, labels):
        self.data, self.labels = data, labels
    def ____(self, idx):
        return self.data[idx], self.labels[idx]
```

CORRECT TOKEN(S): `__getitem__`

Technical Breakdown & Key Insight: Requires double underscores on both sides: __getitem__. Enables dataset[idx] syntax for DataLoader workers.

QUESTION 10 • PyTorch Dataset Length — 16 Points

In PyTorch, which magic method must be implemented inside a custom Dataset class to return the total size (number of samples) in the dataset?

```
class CustomDataset(Dataset):
    def __init__(self, data):
        self.data = data
    def ____(self):
        return len(self.data)
```

CORRECT TOKEN(S): `__len__`

Technical Breakdown & Key Insight: Requires double underscores: __len__. DataLoader uses this to compute the number of batches per training epoch.

QUESTION 11 • PyTorch State Dict (Saving) **16 Points**

In PyTorch, which method returns a Python dictionary containing the entire mapping of learnable parameter tensors (weights and biases) of a model for saving to disk?

```
torch.save(model.____(), "model_weights.pth")
```

CORRECT TOKEN(S): `state_dict (or state_dict())`

Technical Breakdown & Key Insight: Best practice saves only learned parameters via model.state_dict() rather than pickling the full class architecture.

QUESTION 12 • PyTorch Loading State Dict **16 Points**

In PyTorch, which method is used to load a previously saved parameter state dictionary into an instantiated model architecture?

```
model = LeNet()
weights = torch.load("model_weights.pth")
model.____(weights)
```

CORRECT TOKEN(S): `load_state_dict`

Technical Breakdown & Key Insight: Remember the underscores: load_state_dict. It restores all learned weights into the instantiated network object.

QUESTION 13 • PyTorch Classification Loss **16 Points**

Which standard PyTorch loss function in torch.nn is used for multi-class classification, implicitly combining LogSoftmax and Negative Log-Likelihood (NLLLoss)?

```
import torch.nn as nn
criterion = nn.____()
```

CORRECT TOKEN(S): `CrossEntropyLoss`

Technical Breakdown & Key Insight: Do NOT add Softmax to your model output when using nn.CrossEntropyLoss; it already computes LogSoftmax internally.

QUESTION 14 • PyTorch DataLoader Class **16 Points**

Which class from the torch.utils.data module is used to combine a dataset with batching, shuffling, and multi-threaded data loading?

```
from torch.utils.data import Dataset, ____
loader = DataLoader(custom_dataset, batch_size=32, shuffle=True)
```

CORRECT TOKEN(S): `DataLoader`

Technical Breakdown & Key Insight: PascalCase: DataLoader. It handles batching, memory pinning, shuffling, and multi-worker background loading.

QUESTION 15 • PyTorch Scalar Extraction **16 Points**

In PyTorch, which method is used to extract a standard Python floating-point scalar value from a 1-element loss tensor for logging?

```
loss = criterion(outputs, labels)
total_loss += loss.____()
```

CORRECT TOKEN(S): `item (or item())`

Technical Breakdown & Key Insight: Always use loss.item() when logging loss! Calling total_loss += loss retains computation graphs, leaking memory.

Category 2: Huawei MindSpore Framework & Ascend Coding (Q16 – Q25)

QUESTION 16 • MindSpore Cell Base Class — 16 Points

In Huawei's MindSpore framework, which base class must all neural network models and functional layers inherit from (equivalent to PyTorch's nn.Module)?

```
import mindspore.nn as nn
class LeNet5(nn.____):
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5)
```

CORRECT TOKEN(S): `Cell`

Technical Breakdown & Key Insight: In MindSpore, the foundational class is nn.Cell (capital C). In PyTorch, it is nn.Module.

QUESTION 17 • MindSpore Construct Method — 16 Points

In Huawei MindSpore, which method must be implemented inside a custom nn.Cell to define the forward computational process (equivalent to PyTorch's forward method)?

```
class Network(nn.Cell):
    def __init__(self):
        super().__init__()
        self.dense = nn.Dense(10, 2)
    def ____(self, x):
        return self.dense(x)
```

CORRECT TOKEN(S): `construct`

Technical Breakdown & Key Insight: Classic MindSpore vs PyTorch distinction: MindSpore uses construct, whereas PyTorch uses forward!

QUESTION 18 • MindSpore High-Level Model API — 16 Points

In MindSpore, which high-level class encapsulates a network, a loss function, and an optimizer to provide high-level training and evaluation interfaces?

```
import mindspore as ms
model = ms.train.____(network, loss_fn, optimizer, metrics={'acc'})
```

CORRECT TOKEN(S): `Model`

Technical Breakdown & Key Insight: Capitalized: Model. Encapsulates training and inference loops into simple model.train() and model.eval() calls.

QUESTION 19 • MindSpore Model Train Execution — 16 Points

In MindSpore, which method of the Model class is invoked to start model training over a specified number of epochs?

```
model = ms.train.Model(network, loss_fn, optimizer)
model.____(epoch=10, train_dataset=ds_train)
```

CORRECT TOKEN(S): `train`

Technical Breakdown & Key Insight: model.train(epoch, dataset) automatically handles iteration loops, gradient calculation, and metrics monitoring.

QUESTION 20 • MindSpore Model Eval Execution **16 Points**

In MindSpore, which method of the Model class evaluates the performance of the trained network against a validation/test dataset?

```
results = model.____(valid_dataset)
print(results)
```

CORRECT TOKEN(S): `eval`

Technical Breakdown & Key Insight: model.eval(dataset) performs inference on the evaluation dataset and returns the registered dictionary of metric scores.

QUESTION 21 • MindSpore Saving Checkpoint **16 Points**

In MindSpore, which function is used to save the network's learned parameters into a checkpoint file on disk?

```
import mindspore as ms
ms.____(network, "model_checkpoint.ckpt")
```

CORRECT TOKEN(S): `save_checkpoint`

Technical Breakdown & Key Insight: Function name: ms.save_checkpoint(network, path). Checkpoint files use the .ckpt Protocol Buffers format.

QUESTION 22 • MindSpore Loading Checkpoint **16 Points**

In MindSpore, which function loads a parameter dictionary from a saved .ckpt checkpoint file?

```
import mindspore as ms
param_dict = ms.____("model_checkpoint.ckpt")
```

CORRECT TOKEN(S): `load_checkpoint`

Technical Breakdown & Key Insight: ms.load_checkpoint(path) parses the .ckpt binary and returns a Python dictionary of parameter names and tensor values.

QUESTION 23 • MindSpore Load Param Into Net **16 Points**

In MindSpore, which function takes a loaded parameter dictionary and loads the weights into the neural network instance?

```
param_dict = ms.load_checkpoint("model.ckpt")
ms.____(network, param_dict)
```

CORRECT TOKEN(S): `load_param_into_net`

Technical Breakdown & Key Insight: Full function name: ms.load_param_into_net(network, param_dict). Restores trained weights into the network instance.

QUESTION 24 • MindSpore Device Target Context **16 Points**

In MindSpore, to run model training on Huawei's dedicated AI processors, what string must be assigned to device_target in set_context?

```
import mindspore as ms
ms.set_context(device_target="____")
```

CORRECT TOKEN(S): `Ascend`

Technical Breakdown & Key Insight: Must be capitalized: Ascend. Options for device_target include 'Ascend', 'GPU', and 'CPU'.

QUESTION 25 • MindSpore Tensor Class **16 Points**

In MindSpore, what is the core multi-dimensional data structure class used to hold model weights and input arrays?

```
import mindspore as ms
import numpy as np
data = np.array([1.0, 2.0, 3.0], dtype=np.float32)
ms_tensor = ms.____(data, dtype=ms.float32)
```

CORRECT TOKEN(S): `Tensor`

Technical Breakdown & Key Insight: Capitalized: Tensor. MindSpore Tensors can be initialized directly from Python lists, NumPy arrays, or scalar values.

Category 3: Scikit-Learn Traditional Machine Learning Coding (Q26 – Q35)

QUESTION 26 • Scikit-Learn Model Fitting **16 Points**

In scikit-learn, which core method is called on an estimator instance to train the machine learning algorithm on feature matrix X and target vector y?

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.____(X_train, y_train)
```

CORRECT TOKEN(S): `fit`

Technical Breakdown & Key Insight: Universal across scikit-learn: fit(X, y) executes model learning for both supervised and unsupervised estimators.

QUESTION 27 • Scikit-Learn Model Prediction **16 Points**

In scikit-learn, which method is used after training to generate predicted target labels or values for new unseen feature samples?

```
lr.fit(X_train, y_train)
predictions = lr.____(X_test)
```

CORRECT TOKEN(S): `predict`

Technical Breakdown & Key Insight: Universal across scikit-learn: predict(X_test) returns class predictions for classifiers or scalar values for regressors.

QUESTION 28 • Scikit-Learn Probability Prediction **16 Points**

In scikit-learn classification models (such as LogisticRegression), which method returns the calibrated class probability estimates for each sample?

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(X_train, y_train)
probabilities = clf.____(X_test)
```

CORRECT TOKEN(S): `predict_proba`

Technical Breakdown & Key Insight: Notice the underscore: predict_proba. Returns an array of shape *(n_samples, n_classes)* with values between 0 and 1.

QUESTION 29 • Scikit-Learn Fit Transform **16 Points**

In scikit-learn preprocessing (e.g., StandardScaler), which combined method calculates transformation statistics on training data and applies scaling in a single step?

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.____(X_train)
```

CORRECT TOKEN(S): `fit_transform`

Technical Breakdown & Key Insight: fit_transform is strictly for training data: it computes mean/variance AND transforms the feature matrix simultaneously.

QUESTION 30 • Scikit-Learn Transform Only **16 Points**

In scikit-learn preprocessing, which method scales testing data using the previously computed training set statistics without re-estimating parameters?

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.____(X_test)
```

CORRECT TOKEN(S): `transform`

Technical Breakdown & Key Insight: Critical Data Leakage Rule: Never use fit_transform on test data! Always use scaler.transform(X_test).

QUESTION 31 • Scikit-Learn Train Test Split **16 Points**

In scikit-learn, which function from model_selection is commonly used to randomly partition a dataset into training and testing subsets?

```
from sklearn.model_selection import ____
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

CORRECT TOKEN(S): `train_test_split`

Technical Breakdown & Key Insight: Imported from sklearn.model_selection: train_test_split randomly divides data according to test_size.

QUESTION 32 • Linear Regression Coef Attribute **16 Points**

In scikit-learn's LinearRegression, which attribute stores the learned weight coefficients (slopes) of the model after fitting?

```
lr = LinearRegression().fit(X, y)
weights = lr.____
```

CORRECT TOKEN(S): `coef_`

Technical Breakdown & Key Insight: Trailing underscore convention: coef_. In scikit-learn, all attributes learned during fit() end with an underscore.

QUESTION 33 • Linear Regression Intercept Attribute **16 Points**

In scikit-learn's LinearRegression, which attribute stores the learned bias (intercept) of the model after fitting?

```
lr = LinearRegression().fit(X, y)
bias = lr.____
```

CORRECT TOKEN(S): `intercept_`

Technical Breakdown & Key Insight: Trailing underscore convention: intercept_. Represents the constant bias term (the y-intercept where X=0).

QUESTION 34 • K-Means Cluster Count Parameter **16 Points**

In scikit-learn's KMeans clustering estimator, which hyperparameter specifies the number of clusters to form?

```
from sklearn.cluster import KMeans
kmeans = KMeans(____=3, random_state=42)
```

CORRECT TOKEN(S): `n_clusters`

Technical Breakdown & Key Insight: Hyperparameter name: n_clusters (not k, not num_clusters). Corresponds to the parameter K in K-Means clustering.

QUESTION 35 • K-Means Cluster Centroids Attribute **16 Points**

In scikit-learn's KMeans estimator, which attribute stores the final coordinates of cluster centers after training?

```
kmeans = KMeans(n_clusters=3).fit(X)
centers = kmeans.____
```

CORRECT TOKEN(S): `cluster_centers_`

Technical Breakdown & Key Insight: Trailing underscore: cluster_centers_. Returns an array of coordinates for the K learned cluster centers.

Category 4: NumPy & Tensor Preprocessing Foundations (Q36 – Q40)

QUESTION 36 • NumPy Zeros Creation **16 Points**

In NumPy, which function initializes an array of given shape and type populated entirely with zeros?

```
import numpy as np
zero_matrix = np.____((3, 3))
```

CORRECT TOKEN(S): `zeros`

Technical Breakdown & Key Insight: np.zeros((shape)) creates an array of zeros. Useful for allocating accumulator buffers and weight matrices.

QUESTION 37 • NumPy Ones Creation **16 Points**

In NumPy, which function initializes an array of given shape and type populated entirely with ones?

```
import numpy as np
ones_matrix = np.____((4, 2))
```

CORRECT TOKEN(S): `ones`

Technical Breakdown & Key Insight: np.ones((shape)) creates an array populated with ones. Frequently used to initialize bias vectors or uniform masks.

QUESTION 38 • NumPy Reshape Function **16 Points**

In NumPy, which method gives a new shape to an array without changing its underlying data elements?

```
arr = np.arange(10)  # Shape (10,)
matrix = arr.____((2, 5))
```

CORRECT TOKEN(S): `reshape`

Technical Breakdown & Key Insight: arr.reshape(new_shape) changes the tensor dimensions while keeping the total number of elements identical.

QUESTION 39 • NumPy Array Shape Attribute **16 Points**

In NumPy, which attribute of an ndarray returns a tuple representing the dimensions of the array?

```
arr = np.zeros((3, 4, 5))
dims = arr.____  # Returns (3, 4, 5)
```

CORRECT TOKEN(S): shape

Technical Breakdown & Key Insight: arr.shape is an attribute (no parentheses!). Returns a tuple like (batch_size, channels, height, width).

QUESTION 40 • NumPy Type Casting **16 Points**

In NumPy, which method casts an array to a specified numerical data type (such as np.float32)?

```
int_arr = np.array([1, 2, 3])
float_arr = int_arr.____(np.float32)
```

CORRECT TOKEN(S): astype

Technical Breakdown & Key Insight: arr.astype(dtype) creates a copy of the array cast to the specified type, essential when preparing inputs for deep learning models.

Category 5: Multi-Blank Official PyTorch LeNet, CNN Math & Checkpoints (Q41 – Q50)

QUESTION 41 • Multi-Blank LeNet Architecture (TM p. 355) **16 Points**

Complete the three blanks defining the convolutional, dense, and pooling layers inside the official PyTorch LeNet constructor.

```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.___[1]___(1, 6, kernel_size=5, stride=1, padding=2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1)
        self.fc1 = nn.___[2]___(16*5*5, 120)
        self.pool = nn.___[3]___(kernel_size=2, stride=2)
```

CORRECT TOKEN(S): [1]: Conv2d | [2]: Linear | [3]: AvgPool2d

Technical Breakdown & Key Insight: Verbatim from HCIA-AI V4.0 Training Material p. 355: Conv2d extracts spatial maps, Linear(16*5*5, 120) takes 400 flattened features, and AvgPool2d downsamples.

QUESTION 42 • Multi-Blank LeNet Forward Pass (TM p. 356 & LG p. 53) **16 Points**

Complete the two blanks in the LeNet forward() method to apply pooling and flatten the 16x5x5 feature tensor into a 1D vector for dense classification.

```
def forward(self, x):
    x = self.___[1]___(self.relu(self.conv1(x)))
    x = self.pool(self.sigmoid(self.conv2(x)))
    x = x.___[2]___(-1, 16*5*5)
    x = self.relu(self.fc1(x))
    return self.fc3(self.relu(self.fc2(x)))
```

CORRECT TOKEN(S): [1]: pool | [2]: view

Technical Breakdown & Key Insight: x.view(-1, 16*5*5) reshapes the 4D feature map *(batch, 16, 5, 5)* into *(batch, 400)*; -1 infers batch size dynamically.

QUESTION 43 • Multi-Blank Loss & SGD Optimizer (TM p. 358) **16 Points**

Complete the two blanks to initialize multi-class cross-entropy loss and stochastic gradient descent with momentum for LeNet training.

```
net = LeNet()
criterion = nn.___[1]___()
optimizer = optim.___[2]___(net.parameters(), lr=0.01, momentum=0.9)
```

CORRECT TOKEN(S): [1]: CrossEntropyLoss | [2]: SGD

Technical Breakdown & Key Insight: Official Training Material p. 358 uses nn.CrossEntropyLoss() and optim.SGD(net.parameters(), lr=0.01, momentum=0.9).

QUESTION 44 • Multi-Blank PyTorch Training Loop (TM p. 358 & LG p. 54) **16 Points**

Fill in the three core PyTorch optimization steps in exact execution order inside the mini-batch training loop.

```
for i, data in enumerate(trainloader, 0):
    inputs, labels = data
    optimizer.___[1]___()
    outputs = net(inputs)
    loss = criterion(outputs, labels)
    loss.___[2]___()
    optimizer.___[3]___()
```

CORRECT TOKEN(S): [1]: zero_grad | [2]: backward | [3]: step

Technical Breakdown & Key Insight: Always clear old gradients (zero_grad) before computing backpropagation (backward) and updating parameters (step).

QUESTION 45 • Multi-Blank PyTorch Test Evaluation (LG p. 55) **16 Points**

Complete the two blanks in the CIFAR-10 / MNIST test evaluation block to disable autograd and extract predicted class indices.

```
correct, total = 0, 0
with torch.___[1]___():
    for images, labels in testloader:
        outputs = net(images)
        _, predicted = torch.___[2]___(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

CORRECT TOKEN(S): [1]: no_grad | [2]: max

Technical Breakdown & Key Insight: torch.no_grad() disables gradient computation; torch.max(outputs.data, 1) returns *(max_values, predicted_indices)* along class dimension 1.

QUESTION 46 • Multi-Blank Model Persistence & Eval (TM p. 359) **16 Points**

Complete the two blanks to save only the trained network parameters dictionary and switch the reloaded model into inference mode.

```
torch.save(net.___[1]___(), 'lenet_mnist.pth')
net = LeNet()
net.load_state_dict(torch.load('lenet_mnist.pth'))
net.___[2]___()
```

CORRECT TOKEN(S): [1]: state_dict | [2]: eval

Technical Breakdown & Key Insight: Recommended PyTorch pattern (TM p. 359): save state_dict(), instantiate LeNet(), load weights, and call net.eval() before inference.

QUESTION 47 • Multi-Blank Transforms & DataLoader (TM p. 357) **16 Points**

Complete the two blanks to convert MNIST PIL images into normalized tensors and wrap the dataset into shuffled batches of 64.

```
transform = transforms.Compose([
    transforms.___[1]___(),
    transforms.Normalize((0.1307,), (0.3081,))
])
trainloader = torch.utils.data.DataLoader(trainset, ___[2]___=64, shuffle=True)
```

CORRECT TOKEN(S): [1]: ToTensor | [2]: batch_size

Technical Breakdown & Key Insight: transforms.ToTensor() converts image pixels to [0.0, 1.0] tensors; batch_size=64 controls samples per mini-batch iteration.

QUESTION 48 • Multi-Blank CNN Dimension & Kernel Math (TM p. 356) **16 Points**

Calculate the numeric values for CNN output spatial width and convolution kernel channel depth.

```
# 1) Input 32x32, kernel 5x5, stride=1, padding=0 -> Output = floor((32-5+0)/1)+1 = ___[1]___
# 2) Input feature map 28x28x64, 128 kernels of size 3x3 -> Each kernel depth = ___[2]___
```

CORRECT TOKEN(S): [1]: 28 | [2]: 64

Technical Breakdown & Key Insight: Output size = (32 - 5)/1 + 1 = **28**. A convolution kernel's channel depth ALWAYS matches the input map's depth (**64**), independent of filter count (128).

QUESTION 49 • Multi-Blank Numerical Deep Learning Constants (TM pp. 218, 235) **16 Points**

Enter the exact numerical constants for the maximum derivative of Sigmoid and total probability sum of Softmax.

```
# Max derivative of Sigmoid sigma'(0) = sigma(0)*(1 - sigma(0)) = ___[1]___
# Sum of all output class probabilities from Softmax layer    = ___[2]___
```

CORRECT TOKEN(S): [1]: 0.25 (or 1/4) | [2]: 1

Technical Breakdown & Key Insight: At x=0, Sigmoid(0)=0.5, so max derivative is 0.5*(1-0.5) = **0.25** (causing vanishing gradients). Softmax outputs form a distribution summing to **1**.

QUESTION 50 • Multi-Blank Official Model File Extensions (TM p. 329) **16 Points**

Fill in the three official model file extensions matching the HCIA-AI V4.0 Model File specification table.

```
# Protocol Buffers checkpoint to resume interrupted training:  model.___[1]___
# Default PyTorch format saving model structure & parameters:  model.___[2]___
# Universal cross-framework middleware (e.g. for TensorRT):    model.___[3]___
```

CORRECT TOKEN(S): [1]: ckpt | [2]: pth (or pt) | [3]: onnx

Technical Breakdown & Key Insight: Per TM p. 329: .ckpt uses Protocol Buffers for resume/fine-tuning; .pt/.pth is default PyTorch; .onnx enables framework migration; .bin is generic binary.

HCIA-AI V4.0 Code & Math Blanks: Quick Reference (v1.1)

Complete review table of all 50 code completion tokens, multi-blank workflows, CNN math constants, and Model File extensions.

Q#	Category	Target Code / Concept Context	Accepted Token(s)
Q1	PyTorch Autograd	In the PyTorch training loop, which method must be calle...	**zero_grad (or zero_grad())**
Q2	PyTorch Backpropagation	In PyTorch, which method is invoked on the computed scal...	**backward (or backward())**
Q3	PyTorch Optimizer Update	In PyTorch, which method of the optimizer is called to u...	**step (or step())**
Q4	PyTorch Model Inheritance	In PyTorch, all user-defined neural network architecture...	**Module**
Q5	PyTorch Forward Method	In PyTorch, which core method must be implemented inside...	**forward**
Q6	PyTorch Inference Mode	During model evaluation and testing in PyTorch, which co...	**no_grad (or no_grad())**
Q7	PyTorch Evaluation Mode	In PyTorch, which method sets the neural network into ev...	**eval (or eval())**
Q8	PyTorch Training Mode	In PyTorch, which method sets the neural network back in...	**train (or train())**
Q9	PyTorch Dataset Indexing	In PyTorch, when building a custom Dataset by subclassin...	**__getitem__**
Q10	PyTorch Dataset Length	In PyTorch, which magic method must be implemented insid...	**__len__**
Q11	PyTorch State Dict (Saving)	In PyTorch, which method returns a Python dictionary con...	**state_dict (or state_dict())**
Q12	PyTorch Loading State Dict	In PyTorch, which method is used to load a previously sa...	**load_state_dict**
Q13	PyTorch Classification Loss	Which standard PyTorch loss function in torch.nn is used...	**CrossEntropyLoss**
Q14	PyTorch DataLoader Class	Which class from the torch.utils.data module is used to ...	**DataLoader**
Q15	PyTorch Scalar Extraction	In PyTorch, which method is used to extract a standard P...	**item (or item())**
Q16	MindSpore Cell Base Class	In Huawei's MindSpore framework, which base class must a...	**Cell**
Q17	MindSpore Construct Method	In Huawei MindSpore, which method must be implemented in...	**construct**
Q18	MindSpore High-Level Model API	In MindSpore, which high-level class encapsulates a netw...	**Model**
Q19	MindSpore Model Train Execution	In MindSpore, which method of the Model class is invoked...	**train**
Q20	MindSpore Model Eval Execution	In MindSpore, which method of the Model class evaluates ...	**eval**
Q21	MindSpore Saving Checkpoint	In MindSpore, which function is used to save the network...	**save_checkpoint**
Q22	MindSpore Loading Checkpoint	In MindSpore, which function loads a parameter dictionar...	**load_checkpoint**
Q23	MindSpore Load Param Into Net	In MindSpore, which function takes a loaded parameter di...	**load_param_into_net**
Q24	MindSpore Device Target Context	In MindSpore, to run model training on Huawei's dedicate...	**Ascend**
Q25	MindSpore Tensor Class	In MindSpore, what is the core multi-dimensional data st...	**Tensor**
Q26	Scikit-Learn Model Fitting	In scikit-learn, which core method is called on an estim...	**fit**

Q#	Category	Target Code / Concept Context	Accepted Token(s)
Q27	Scikit-Learn Model Prediction	In scikit-learn, which method is used after training to ...	**predict**
Q28	Scikit-Learn Probability Prediction	In scikit-learn classification models (such as LogisticR...	**predict_proba**
Q29	Scikit-Learn Fit Transform	In scikit-learn preprocessing (e.g., StandardScaler), wh...	**fit_transform**
Q30	Scikit-Learn Transform Only	In scikit-learn preprocessing, which method scales testi...	**transform**
Q31	Scikit-Learn Train Test Split	In scikit-learn, which function from model_selection is ...	**train_test_split**
Q32	Linear Regression Coef Attribute	In scikit-learn's LinearRegression, which attribute stor...	**coef_**
Q33	Linear Regression Intercept Attribute	In scikit-learn's LinearRegression, which attribute stor...	**intercept_**
Q34	K-Means Cluster Count Parameter	In scikit-learn's KMeans clustering estimator, which hyp...	**n_clusters**
Q35	K-Means Cluster Centroids Attribute	In scikit-learn's KMeans estimator, which attribute stor...	**cluster_centers_**
Q36	NumPy Zeros Creation	In NumPy, which function initializes an array of given s...	**zeros**
Q37	NumPy Ones Creation	In NumPy, which function initializes an array of given s...	**ones**
Q38	NumPy Reshape Function	In NumPy, which method gives a new shape to an array wit...	**reshape**
Q39	NumPy Array Shape Attribute	In NumPy, which attribute of an ndarray returns a tuple ...	**shape**
Q40	NumPy Type Casting	In NumPy, which method casts an array to a specified num...	**astype**
Q41	Multi-Blank LeNet Architecture (TM p. 355)	Complete the three blanks defining the convolutional, de...	**[1]: Conv2d | [2]: Linear | [3]: AvgPool2d**
Q42	Multi-Blank LeNet Forward Pass (TM p. 356 & LG p. 53)	Complete the two blanks in the LeNet forward() method to...	**[1]: pool | [2]: view**
Q43	Multi-Blank Loss & SGD Optimizer (TM p. 358)	Complete the two blanks to initialize multi-class cross-...	**[1]: CrossEntropyLoss | [2]: SGD**
Q44	Multi-Blank PyTorch Training Loop (TM p. 358 & LG p. 54)	Fill in the three core PyTorch optimization steps in exa...	**[1]: zero_grad | [2]: backward | [3]: step**
Q45	Multi-Blank PyTorch Test Evaluation (LG p. 55)	Complete the two blanks in the CIFAR-10 / MNIST test eva...	**[1]: no_grad | [2]: max**
Q46	Multi-Blank Model Persistence & Eval (TM p. 359)	Complete the two blanks to save only the trained network...	**[1]: state_dict | [2]: eval**
Q47	Multi-Blank Transforms & DataLoader (TM p. 357)	Complete the two blanks to convert MNIST PIL images into...	**[1]: ToTensor | [2]: batch_size**
Q48	Multi-Blank CNN Dimension & Kernel Math (TM p. 356)	Calculate the numeric values for CNN output spatial widt...	**[1]: 28 | [2]: 64**
Q49	Multi-Blank Numerical Deep Learning Constants (TM pp. 218, 235)	Enter the exact numerical constants for the maximum deri...	**[1]: 0.25 (or 1/4) | [2]: 1**
Q50	Multi-Blank Official Model File Extensions (TM p. 329)	Fill in the three official model file extensions matchin...	**[1]: ckpt | [2]: pth (or pt) | [3]: onnx**